summary_components Module Manual

MyCodingRiver_2016_2026

Generated: 2026-08-18

This manual explains the purpose, structure, interfaces, build flow, and expected integration pattern of the summary_components folder. The code is organized as a testable C component summary for a speech interaction pipeline. It is not a complete production speech client or server; it is a clean reference layer that captures legacy system concepts behind small interfaces and unit tests.

Purpose

The summary_components folder groups old speech-client and speech-server engineering ideas into three clear C modules:

- core_engineering: low-level Linux/C helpers.
- audio_speech_processing: audio format, device, VAD, voice, and ASR interface examples.
- business_interaction: packet protocol, HTTP request generation, JSON extraction, and command routing examples.

The practical function of this folder is to provide a readable and testable map of a voice assistant style system:

```
Microphone
  -> ALSA/tinyalsa record
  -> voice preprocessing
  -> VAD speech detection
  -> PCM or Opus packet
  -> TCP post_content protocol
  -> UniSound ASR
  -> text result
  -> command server HTTP/TCP request
  -> JSON command response
  -> TTS / music / media control action
  -> MP3/audio playback
```

Directory Layout

The folder layout is intentionally responsibility-based:

```
summary_components
|-- ARCHITECTURE_OVERVIEW.md
|-- core_engineering
|   |-- include/core_interfaces.h
|   |-- source
|   |-- tests/test_core_interfaces.c
|   ‘-- Makefile
|-- audio_speech_processing
|   |-- include/audio_interfaces.h
|   |-- source
|   |-- tests/test_audio_interfaces.c
|   ‘-- Makefile
‘-- business_interaction
    |-- include/business_interfaces.h
    |-- source
    |-- tests/test_business_interfaces.c
    ‘-- Makefile
```

The intended dependency direction is:

```
core_engineering
  -> audio_speech_processing
  -> business_interaction
  -> speech client / speech server application
```

The modules are currently independent for testing. The dependency direction documents the intended application architecture, not a strict linker dependency in the current examples.

Build and Test

Each module has its own Makefile. Build and run each test binary from the module directory:

```
cd core_engineering
make run

cd ../audio_speech_processing
make run

cd ../business_interaction
make run
```

The tests use simple C assertions implemented inside the test source files. No third-party C test framework is required.

Expected success messages:

```
core_engineering tests passed
audio_speech_processing tests passed
business_interaction tests passed
```

General Coding Conventions

All three modules use a small C-style status convention:

- Success is zero: CORE_OK, AUDIO_OK, or BUSINESS_OK.
- Failure is negative one: CORE_ERROR, AUDIO_ERROR, or BUSINESS_ERROR.
- Most functions validate pointers and sizes before using caller-provided data.
- External systems are wrapped by small interfaces, and several are implemented as mocks for testability.

The code follows an old-server compatible style: plain C structs, explicit buffer sizes, direct return codes, and dependency-free test programs.

core_engineering Module

Responsibility

The core_engineering module captures low-level system utilities that a speech server or client normally needs:

- TCP server socket setup, accept, send-all, and close.
- UDP server socket setup.
- pthread mutex protected counter.
- Log rotation decision helpers.
- Process monitor decision helpers.

Files

```
core_engineering/include/core_interfaces.h
core_engineering/source/core_socket_example.c
```

```
core_engineering/source/core_thread_example.c
core_engineering/source/core_log_example.c
core_engineering/source/core_process_monitor_example.c
core_engineering/tests/test_core_interfaces.c
```

Main Data Structures

struct core_tcp_server
Stores a TCP socket file descriptor, bound port, and listen backlog.

struct core_udp_server
Stores a UDP socket file descriptor and bound port.

struct core_thread_counter
Stores a pthread mutex and an integer counter protected by that mutex.

struct core_log_config
Stores log file name, log path, max size, and number of backups to retain.

struct core_monitor_status
Stores expected and current process counts for monitor decisions.

API Reference

core_is_valid_port(port)
Returns nonzero when the port is in the valid explicit TCP/UDP range, 1 through 65535.

core_tcp_server_open(server, port)
Validates the port, creates a TCP socket, binds to the port, and starts listen with CORE_TCP_BACKLOG.

core_tcp_server_accept(server)
Accepts one TCP client from an opened server socket and returns the client file descriptor.

core_tcp_send_all(fd, buffer, length)
Sends all requested bytes to a connected TCP peer. Returns CORE_OK on success.

core_socket_close(fd_pointer)
Closes a valid socket descriptor and resets it to an invalid value.

core_udp_server_open(server, port)
Validates the port, creates a UDP socket, and binds it to the requested port.

core_thread_counter_init(counter)
Initializes the mutex and sets the counter value to zero.

core_thread_counter_add(counter, delta)
Locks the mutex, adds delta, unlocks the mutex, and returns CORE_OK on success.

core_thread_counter_get(counter)
Reads the counter value with mutex protection.

core_thread_counter_destroy(counter)
Destroys the counter mutex.

core_log_need_rotate(config, current_size)
Returns one when current_size is greater than or equal to config->max_size. Returns zero when rotation is not needed.

core_log_format_backup_name(config, index, out, out_size)
Formats a backup log name using the configured path and file name, for example:

```
logSystem/Server_Runing.log~3.log
```

core_monitor_decide(status)
Returns CORE_MONITOR_START when current_count is zero, CORE_MONITOR_KEEP when current_count equals expect_count, and CORE_MONITOR_KILL_EXTRA when current_count is greater than expect_count.

Implementation Notes

The socket code is Linux/POSIX oriented. The source includes a Windows stub branch so interface tests can still compile in environments that do not provide POSIX socket headers.

This module is suitable as a reference for server infrastructure behavior, but production code should add nonblocking I/O handling, timeout strategy, structured logging, and real process discovery.

audio_speech_processing Module

Responsibility

The audio_speech_processing module captures the audio and speech processing stack behind small C interfaces:

- PCM byte calculations and frame splitting.
- WAV PCM header build and parse helpers.
- MP3 and Opus decoder interface examples.
- ALSA/tinyalsa device state interface example.
- Energy-based VAD example.
- Voice processing mock.
- UniSound ASR session lifecycle mock.

Files

```
audio_speech_processing/include/audio_interfaces.h
audio_speech_processing/source/audio_pcm_format.c
audio_speech_processing/source/audio_wav_format.c
audio_speech_processing/source/audio_mp3_format.c
audio_speech_processing/source/audio_opus_format.c
audio_speech_processing/source/audio_alsa_device.c
audio_speech_processing/source/audio_vad_voice.c
audio_speech_processing/source/audio_unisound_asr.c
audio_speech_processing/tests/test_audio_interfaces.c
```

Main Data Structures

struct audio_pcm_format
: Describes sample rate, channel count, and bits per sample.

struct audio_pcm_split_state
: Stores remaining bytes that did not fill a complete 640-byte UniSound feed chunk.

struct audio_wav_header
: Models a basic 44-byte PCM WAV header.

struct audio_mp3_decoder
: Stores mock MP3 decoder state, sample rate, and channels.

struct audio_opus_decoder
: Stores mock Opus decoder state, sample rate, and channels.

struct audio_alsa_device
: Stores mock ALSA device name, mode, PCM format, and started state.

struct audio_vad_state
: Stores speech active state, start and end thresholds, silence frame limit, and current silence count.

struct audio_unisound_session
: Stores mock UniSound session state, credentials, fed byte count, and recognition result.

API Reference

audio_pcm_bytes_per_sample(format)
: Returns the byte width for one PCM sample in one channel. For 16-bit PCM this returns 2.

audio_pcm_frame_bytes(format, sample_count)
Returns sample_count * channels * bytes_per_sample.

audio_pcm_split_to_640(state, input, input_length, chunk_func, user_data)
Combines previous rest bytes with new input, emits full 640-byte chunks through the callback, and stores trailing bytes for the next call. This matches the legacy UniSound feed pattern.

audio_pcm_12mic_to_float(input, voice_out, aec_out, mic_count, sample_count)
Converts interleaved 12-microphone short samples into normalized float buffers for voice and AEC style processing.

audio_wav_build_header(header, format, pcm_bytes)
Builds a basic RIFF/WAVE PCM header and calculates byte rate, block align, file size, and data size.

audio_wav_parse_format(header, format, pcm_bytes)
Validates a basic WAV PCM header and extracts sample rate, channels, bit depth, and PCM byte count.

audio_mp3_decoder_open(decoder, sample_rate, channels)
Initializes a mock MP3 decoder state.

audio_mp3_decode_mock(decoder, mp3_data, mp3_length, pcm_out, pcm_capacity)
Validates input and writes deterministic mock PCM samples. It does not call libmad or another real MP3 decoder.

audio_mp3_decoder_close(decoder)
Clears mock MP3 decoder state.

audio_opus_decoder_open(decoder, sample_rate, channels)
Initializes a mock Opus decoder state.

audio_opus_decode_mock(decoder, packet, packet_length, pcm_out, max_samples)
Validates input and writes deterministic mock PCM samples. It does not call libopus.

audio_opus_decoder_close(decoder)
Clears mock Opus decoder state.

audio_alsa_open_example(device, device_name, mode, format)
Initializes mock ALSA/tinyalsa device state for record or playback mode.

audio_alsa_start_example(device)
Marks the mock device as started.

audio_alsa_stop_example(device)
Marks the mock device as stopped.

audio_vad_init(state, start_score, end_score, end_frame_count)
Initializes the energy-based VAD state.

audio_vad_feed_pcm(state, pcm, sample_count)
Computes average absolute PCM energy. Returns silence, speech, or end-of-speech according to configured thresholds.

audio_voice_process_mock(input, output, sample_count, gain)
Converts normalized float samples to 16-bit PCM with gain and clipping.

audio_unisound_init_example(session, app_key, secret_key)
Initializes a mock UniSound ASR session and stores credentials in the session object.

audio_unisound_start_example(session)
Marks the mock recognizer as started.

audio_unisound_feed_example(session, pcm, pcm_length)
Adds PCM bytes to the mock recognizer input count.

audio_unisound_stop_example(session)
Stops the mock recognizer and produces a deterministic mock recognition result.

audio_unisound_get_result_example(session)
Returns the mock recognition result string.

Audio Stack Mapping

The module maps legacy technology names into isolated interfaces:

```
ALSA/tinyalsa -> record and playback device layer
Voice SDK     -> microphone array preprocessing
Huwen VAD     -> speech start/end detection
Opus          -> compressed network audio
PCM           -> recognition input format
UniSound SDK  -> ASR recognition
MP3/libmad    -> TTS or music playback decode
```

Implementation Notes

This module deliberately avoids linking real external audio libraries in the tests. The mock interfaces allow the code to compile and verify lifecycle, buffer, and routing logic without SDK or hardware dependencies.

Production replacement points are clear: replace the mock MP3, Opus, ALSA, Voice, VAD, and UniSound implementations with adapters around the real libraries while preserving the public interface shape where useful.

business_interaction Module

Responsibility

The business_interaction module captures the network and command layer:

- Binary post_content packet packing, unpacking, and validation.
- HTTP GET and POST request text builders.
- Baidu TTS style HTTP request generation.
- Lightweight JSON string extraction.
- Command JSON routing to media, TTS, music, and volume actions.

Files

```
business_interaction/include/business_interfaces.h
business_interaction/source/business_protocol.c
business_interaction/source/business_http.c
business_interaction/source/business_json.c
business_interaction/source/business_command.c
business_interaction/tests/test_business_interfaces.c
```

Main Data Structures

struct business_post_content
: Represents the binary packet exchanged between client and speech server. It contains contType, contLength, and a fixed 2048-byte payload buffer.

struct business_http_request
: Stores host, port, and generated HTTP request text.

struct business_command_result
: Stores parsed command action, text payload, and optional media URL.

API Reference

business_protocol_pack(packet, cont_type, payload, payload_length)
: Validates packet type and length, clears the packet, stores type and length, and copies payload bytes.

business_protocol_unpack(packet, cont_type, out, out_size)
: Validates the packet and copies payload bytes to the caller buffer. Returns copied payload length.

business_protocol_validate(packet)
Checks that packet type is start, data, or stop and that payload length fits the fixed buffer.

business_http_build_get(request, host, path)
Builds a simple HTTP/1.0 GET request string.

business_http_build_post(request, host, path, body)
Builds a simple HTTP/1.0 POST request string with Content-Length.

business_http_build_baidu_tts(request, text)
URL-encodes text for the supported subset and builds a Baidu text2audio style GET request.

business_json_get_string(json, key, out, out_size)
Extracts a simple JSON string value by key using lightweight string scanning.

business_json_has_token(json, token)
Returns nonzero when the JSON text contains the requested token.

business_command_parse_json(json, result)
Routes recognized JSON snippets to one of the BUSINESS_ACTION values.

Command Routing

The command parser maps response content to action constants:

```
resultMusic or songUrl       -> BUSINESS_ACTION_MUSIC_PLAY
"cmdType":"chat" or chatAnswer -> BUSINESS_ACTION_CHAT_TTS
mediaPause                   -> BUSINESS_ACTION_MEDIA_PAUSE
mediaResume                  -> BUSINESS_ACTION_MEDIA_RESUME
mediaStop                    -> BUSINESS_ACTION_MEDIA_STOP
mediaVolumeIncrease          -> BUSINESS_ACTION_VOLUME_UP
mediaVolumeDecrease          -> BUSINESS_ACTION_VOLUME_DOWN
unknown command              -> BUSINESS_ACTION_NONE
```

Implementation Notes

The JSON extraction is intentionally lightweight. It is suitable for small examples and deterministic unit tests, but production code should use a real JSON parser such as cJSON or another structured parser. A real parser is needed for escaped characters, nested objects, arrays, numbers, booleans, null values, malformed input handling, and security hardening.

End-to-End Integration Pattern

A complete application built on the concepts in this folder would normally connect the modules in this sequence:

```
1. Start infrastructure:
   - configure logging
   - start process monitor
   - open TCP or UDP server/client sockets

2. Start audio path:
   - open ALSA/tinyalsa record device
   - initialize voice processing and VAD
   - initialize ASR session

3. Capture and detect speech:
   - read PCM frames from microphone
   - convert microphone array samples if needed
   - feed VAD
   - begin packet transfer when speech starts

4. Transfer recognition audio:
   - split PCM into required chunk sizes
   - pack start/data/stop packets
   - send packets over TCP
```

```
5. Recognize and route text:
   - feed PCM into ASR
   - stop ASR and read recognized text
   - send text to command server
   - parse JSON response

6. Execute response:
   - chat answer -> TTS
   - music result -> play URL
   - media control -> pause/resume/stop
   - volume command -> adjust output volume

7. Playback:
   - decode MP3 or other stream when needed
   - send PCM to ALSA/tinyalsa playback device
```

Example Pseudocode

The following pseudocode shows how the interface concepts fit together. It is not a complete source file.

```
struct audio_vad_state vad;
struct audio_unisound_session asr;
struct business_post_content packet;
struct business_command_result command;

audio_vad_init(&vad, 0.10f, 0.04f, 3);
audio_unisound_init_example(&asr, "app-key", "secret-key");
audio_unisound_start_example(&asr);

while (recording) {
    read_pcm_frame(pcm, sample_count);

    vad_state = audio_vad_feed_pcm(&vad, pcm, sample_count);
    if (vad_state == AUDIO_VAD_SPEECH) {
        business_protocol_pack(&packet,
                               BUSINESS_CONT_DATA,
                               (const char *)pcm,
                               pcm_bytes);
        send_packet(packet);
        audio_unisound_feed_example(&asr, (const char *)pcm, pcm_bytes);
    }

    if (vad_state == AUDIO_VAD_END) {
        audio_unisound_stop_example(&asr);
        text = audio_unisound_get_result_example(&asr);
        json = request_command_server(text);
        business_command_parse_json(json, &command);
        execute_command(command);
        break;
    }
}
```

Known Limitations

This folder is useful as a component summary and test harness, but it should not be treated as production-ready without additional work.

- The audio SDK integrations are mocks or examples, not real hardware/codec/ASR integrations.
- The JSON parser is string-scanning only and is not safe for general JSON input.
- HTTP request builders generate request text but do not implement a complete HTTP client.
- Socket helpers are basic blocking examples.
- Fixed-size buffers simplify tests but require careful bounds management in production.

- The manual protocol uses simple integer fields and does not define byte order, framing across architectures, or version negotiation.
- Security features such as TLS, authentication, input sanitization, and robust error recovery are out of scope.

Recommended Production Hardening

If this component model is promoted into a real product, prioritize the following changes:

1. Replace mock SDK code with adapters around the real ALSA/tinyalsa, Opus, MP3, Voice/VAD, and UniSound APIs.
2. Replace lightweight JSON scanning with a structured parser.
3. Define a stable wire protocol: byte order, packet framing, version, checksum or length validation, and error responses.
4. Add timeout, retry, reconnect, and partial-read/partial-write handling for network code.
5. Add structured logs and real log rotation.
6. Move secrets such as ASR app keys out of source and into secure configuration.
7. Add integration tests with real or simulated audio streams.
8. Add sanitizers, static analysis, and CI build targets for the supported platforms.

Quick Reference

Module responsibility summary:

```
core_engineering
  socket, thread, log, process monitor

audio_speech_processing
  PCM, WAV, MP3, Opus, ALSA, VAD, Voice, UniSound ASR

business_interaction
  post_content protocol, HTTP request, JSON parsing, command routing
```

Common constants:

```
CORE_OK / AUDIO_OK / BUSINESS_OK       = 0
CORE_ERROR / AUDIO_ERROR / BUSINESS_ERROR = -1
AUDIO_UNISOUND_CHUNK                   = 640
AUDIO_PCM_BLOCK                        = 2048
BUSINESS_POST_BUFF                     = 2048
BUSINESS_CONT_START                    = 1
BUSINESS_CONT_DATA                     = 2
BUSINESS_CONT_STOP                     = 3
```

Maintenance Notes

Keep headers as the public contract for each module. When changing behavior, update the relevant interface header, source implementation, unit test, architecture overview, and this manual together.